

■ Copa do Mundo FIFA 2026

Estudo Detalhado dos Códigos Python

Explicação linha por linha para iniciantes

Arquivo 1	fifa_estatisticas_jogos.py
Arquivo 2	fifa_ranking_oficial.py
Nível	Iniciante a Intermediário
Data	26 de junho de 2026

Este documento explica em detalhes cada parte dos dois scripts Python criados para extrair e analisar dados da Copa do Mundo FIFA 2026. O objetivo é que você entenda **o que cada linha faz, por que foi escrita assim e como reutilizar esses conceitos** em outros projetos.

ARQUIVO 1 — fifa_estatisticas_jogos.py

Este script baixa os eventos de cada jogo disputado (gols, faltas, chutes...) do endpoint **/timelines** da API da FIFA e gera 4 planilhas: artilharia, faltas, chutes e destaque por jogo.

1. Cabeçalho e importações

Todo script Python começa com as importações — dizemos ao Python quais ferramentas vamos usar. Cada **import** carrega uma biblioteca (conjunto de funções prontas).

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
# ↑ Essas duas linhas dizem:
# 1. Use o Python 3 para rodar este arquivo
# 2. O arquivo usa caracteres UTF-8 (acentos, ç, ã...)

import csv          # lê e escreve arquivos .csv (planilhas)
import json         # lê e escreve arquivos .json (dados brutos)
import os           # acessa o sistema de arquivos (pastas, caminhos)
import re           # expressões regulares (extração de texto)
import time         # funções de tempo (ex: pausar o script)
from collections import defaultdict # dicionário com valor padrão automático
from datetime import datetime, timezone # manipulação de datas e horários
```

■ Bibliotecas como csv, json, os, re, time, collections e datetime já vêm instaladas com o Python. Só o 'requests' precisa ser instalado manualmente com: `pip install requests`

2. Bloco de Configuração

Agrupamos todas as configurações no topo do arquivo. Assim, se precisar mudar algo (ex: o número de jogadores no ranking), você altera aqui sem precisar procurar no meio do código.

```
BASE = "https://api.fifa.com/api/v3" # URL base da API
ID_TEMPORADA = "285023" # ID da Copa 2026 na base da FIFA
ARQUIVO_ENTRADA = "fifa_jogos_completo.json" # JSON gerado pelo extrator
PAUSA = 0.25 # segundos de espera entre requisições
TOP_N = 30 # quantos jogadores salvar em cada CSV

# Tabela de pontos – define o peso de cada tipo de evento no score
PONTOS = {
    0: +4, # Gol (Type 0 na API)
    41: +4, # Gol de pênalti (Type 41)
    34: +3, # Gol contra (Type 34) – peso menor
    1: +2, # Assistência (Type 1)
    12: +1, # Chute a gol (Type 12)
    57: +1, # Gol evitado / defesa (Type 57)
    18: -0.5, # Falta cometida (Type 18)
    2: -1, # Cartão amarelo (Type 2)
    3: -3, # Cartão vermelho (Type 3)
}
```

■ Usar um dicionário para os pontos (em vez de vários if/elif) é mais limpo e fácil de ajustar. Para mudar o peso de um gol de 4 para 5, basta editar a linha '0: +4'.

3. Configuração da Sessão HTTP

Criamos uma **sessão** para todas as requisições. Uma sessão é como abrir o navegador uma vez e manter as configurações para todos os sites que você visita — mais eficiente do que configurar cada requisição separadamente.

```
SESSAO = requests.Session()
# Session() mantém cookies e headers entre requisições – mais eficiente
# do que usar requests.get() direto em cada chamada.
SESSAO.headers.update({
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)...",
    # ↑ Faz o servidor pensar que somos o navegador Chrome no Windows.
    # Sem isso, a API pode rejeitar a requisição.
    "Accept": "application/json, text/plain, */*",
    # ↑ Avisamos que aceitamos resposta em JSON.
    "Origin": "https://inside.fifa.com",
    "Referer": "https://inside.fifa.com/",
    # ↑ Fingimos que a requisição veio do site oficial da FIFA.
    # É o mesmo que o navegador faz automaticamente.
})
```

4. Função: texto()

A FIFA entrega nomes de times, estádios e eventos em múltiplos idiomas dentro de uma lista. Esta função escolhe o idioma desejado.

```
def texto(lista, idioma="pt"):
    # Exemplo do que a API retorna:
    # [ {"Locale": "pt-BR", "Description": "França"},
    #   {"Locale": "en-GB", "Description": "France"} ]
    if not lista:
        return "" # lista vazia → devolve string vazia
    if isinstance(lista, str):
        return lista # já é texto puro → devolve direto
    for item in lista:
        loc = (item.get("Locale") or "").lower()
        # .lower() transforma "PT-BR" em "pt-br" para comparar sem maiúsculas
        if loc == idioma or loc.startswith(idioma):
            # "pt-br".startswith("pt") → True
            return item.get("Description", "")
    # Se não encontrou o idioma, usa o primeiro da lista (fallback)
    return lista[0].get("Description", "")
```

■ O padrão 'or ""' e 'or {}' aparece muito no código. Significa: 'se o valor for None ou vazio, use este valor padrão'. Evita erros quando a API não retorna um campo esperado.

5. Função: nome_jogador() — Regex

O nome do jogador vem embutido dentro do texto do EventDescription, no formato '*NOME (Seleção) fez algo*'. Usamos uma **expressão regular (regex)** para extrair as duas informações.

```
def nome_jogador(evento):
    # Pega o texto da descrição do evento (ex: "Julian QUINONES (México) marca o gol!!")
    desc = texto(evento.get("EventDescription"))
    if not desc:
        return "", "" # evento sem descrição → devolve vazio
    # re.match testa se o texto COMEÇA com o padrão fornecido.
    # Padrão: r"^([^(]+)\(([^(]+)\)"
    #
    # ^          → início da string
    # ([^(]+)    → GRUPO 1: qualquer caractere exceto "(" (o nome do jogador)
    # \(        → um parêntese literal de abertura
    # ([^(]+)    → GRUPO 2: qualquer caractere exceto ")" (o nome do time)
    # \)        → um parêntese literal de fechamento
    m = re.match(r"^([^(]+)\(([^(]+)\)", desc.strip())
    if m:
        nome = m.group(1).strip().title()
        # .strip() remove espaços nas bordas
        # .title() coloca cada palavra com inicial maiúscula
        # "julian quinones" → "Julian Quinones"
        time_nome = m.group(2).strip()
        return nome, time_nome
    return desc[:40], "" # fallback: usa os primeiros 40 caracteres
```

Entrada (desc)	nome retornado	time retornado
Julian QUINONES (México) marca o gol!!	Julian Quinones	México
Kylian MBAPPE (França) assistência	Kylian Mbappe	França
VINICIUS JUNIOR (Brasil) falta sofrida	Vinicius Junior	Brasil

6. Função: ja_disputado()

A API retorna tanto jogos passados quanto futuros. Esta função verifica se a data do jogo já passou comparando com o momento atual.

```
def ja_disputado(jogo):
    # A data do jogo vem assim: "2026-06-11T19:00:00Z"
    # (formato ISO 8601 – padrão internacional)
    iso = jogo.get("Date") or jogo.get("LocalDate")
    # "or" tenta o segundo campo se o primeiro for None/vazio
    if not iso:
        return False # sem data → assume não disputado
    try:
        # Converte a string de data para um objeto datetime
        # [:19] pega só os primeiros 19 caracteres: "2026-06-11T19:00:00"
        d = datetime.strptime(str(iso)[:19], "%Y-%m-%dT%H:%M:%S")
        # datetime.now(timezone.utc) = momento atual em UTC
        # .replace(tzinfo=None) remove o fuso para comparar sem erro
        agora = datetime.now(timezone.utc).replace(tzinfo=None)
        return d < agora # True se o jogo já aconteceu
    except Exception:
        return False # se der qualquer erro na conversão, assume não disputado
```

7. Função: salvar_csv()

Função genérica que salva qualquer lista de dicionários em CSV. Criamos uma função para isso para não repetir o mesmo código 4 vezes.

```
def salvar_csv(caminho, cabecalho, linhas):
    with open(caminho, "w", newline="", encoding="utf-8-sig") as f:
        # "w" → abre para escrita (cria ou sobrescreve)
        # newline="" → necessário no Windows para não duplicar linhas
        # utf-8-sig → UTF-8 com BOM (evita problema de acentos no Excel)
        w = csv.DictWriter(f, fieldnames=cabecalho)
        # DictWriter escreve dicionários como linhas de CSV
        # fieldnames define a ordem das colunas
        w.writeheader() # escreve a linha de cabeçalho (nomes das colunas)
        w.writerows(linhas) # escreve todas as linhas de dados de uma vez
    print(f" Salvo: {caminho}")
```

■ A diferença entre `csv.writer` e `csv.DictWriter`: `writer` recebe listas `['valor1', 'valor2']`, `DictWriter` recebe dicionários `{'coluna': 'valor'}`. `DictWriter` é mais seguro porque usa os nomes das colunas.

8. Função main() — Carregando os dados

A função **main()** é o coração do script. Ela chama todas as outras funções na ordem certa. Vamos estudá-la em partes.

```
def main():
    # os.path.dirname(__file__) → pasta onde este script está salvo
    # os.path.abspath() → converte para caminho absoluto (ex: /Users/lucas/...)
    pasta = os.path.dirname(os.path.abspath(__file__)) or os.getcwd()
    entrada = os.path.join(pasta, ARQUIVO_ENTRADA)
    # os.path.join une pasta + nome do arquivo de forma correta
    # (no Windows usa \ , no Mac/Linux usa /)
    # Verifica se o arquivo JSON existe antes de tentar abrir
    if not os.path.exists(entrada):
        print(f"Arquivo não encontrado: {entrada}")
        return # para a execução aqui

    # Abre e lê o JSON
    with open(entrada, encoding="utf-8") as f:
        dados = json.load(f)
    # json.load() lê o arquivo e converte para lista/dicionário Python
    # O "with" garante que o arquivo é fechado mesmo se der erro
    # Filtra só os jogos da Copa 2026
    jogos_2026 = [j for j in dados if str(j.get("IdSeason")) == ID_TEMPORADA]
    # List comprehension: cria nova lista só com itens que passam na condição
    # str() converte para string porque o ID pode vir como número ou texto
    # Filtra só os já disputados
    disputados = [j for j in jogos_2026 if ja_disputado(j)]
    print(f"{len(disputados)} jogos disputados encontrados.")
```

9. Função main() — Acumuladores com defaultdict

```
# defaultdict(lambda: {...}) cria um dicionário onde cada nova chave
# recebe automaticamente o valor padrão definido pelo lambda.
#
# Sem defaultdict, precisaríamos verificar:
# if nome not in artilharia:
#     artilharia[nome] = {"gols": 0, "time": ""}
# artilharia[nome]["gols"] += 1
#
# Com defaultdict, simplesmente:
# artilharia[nome]["gols"] += 1 ← funciona direto!
artilharia = defaultdict(lambda: {"gols": 0, "time": ""})
faltas_rank = defaultdict(lambda: {"faltas": 0, "time": ""})
chutes_rank = defaultdict(lambda: {"chutes": 0, "time": ""})
destaque_jogos = [] # lista simples para guardar o destaque de cada jogo
```

10. Função main() — Loop pelos jogos

```

for i, jogo in enumerate(disputados):
    # enumerate() retorna (índice, item) – i vai de 0 até len-1
    # Extraí os 4 IDs necessários para montar a URL do /timelines
    idc = jogo.get("IdCompetition") # "17"
    ids = jogo.get("IdSeason")      # "285023"
    idst = jogo.get("IdStage")      # ex: "289273"
    idm = jogo.get("IdMatch")       # ex: "400021443"

    # Informações do jogo para o CSV de destaque
    nc = texto(jogo.get("Home", {}).get("TeamName")) # nome do time casa
    nf = texto(jogo.get("Away", {}).get("TeamName")) # nome do visitante
    gc = jogo.get("Home", {}).get("Score", "?")      # gols casa
    gf = jogo.get("Away", {}).get("Score", "?")      # gols fora
    data = (jogo.get("Date") or "")[:10] # só a data: "2026-06-11"
    if not all([idc, ids, idst, idm]):
        continue # pula jogos com IDs incompletos
    try:
        resp = SESSAO.get(
            f"{BASE}/timelines/{idc}/{ids}/{idst}/{idm}",
            params={"language": "pt"},
            timeout=15, # desiste se demorar mais de 15 segundos
        )
        if not resp.ok:
            continue # pula se a API retornou erro
        eventos = resp.json().get("Event", [])
        # .get("Event", []) → se "Event" não existir, retorna lista vazia
    except Exception as e:
        print(f" Erro no jogo {idm}: {e}")
        continue
    time.sleep(PAUSA) # pausa entre requisições para não sobrecarregar a API
    if (i + 1) % 16 == 0: # a cada 16 jogos, mostra progresso
        print(f" {i + 1}/{len(disputados)} jogos processados...")

```

11. Função main() — Processando os eventos de cada jogo

[illegible]

12. Função main() — Encontrando o destaque do jogo

```
# Após processar todos os eventos do jogo,
# encontra o jogador com maior score
if score_jogo: # se tiver pelo menos um jogador registrado
    dest_nome, dest = max(
        score_jogo.items(),
        key=lambda x: x[1]["score"]
        # max() percorre todos os jogadores e retorna o de maior score
        # x é (nome, dicionário_de_stats)
        # x[1]["score"] acessa o score do dicionário
    )
    destaque_jogos.append({
        "data": data,
        "jogo": f"{nc} {gc} x {gf} {nf}",
        "destaque": dest_nome,
        "time": dest["time"],
        "gols": dest["gols"],
        "assistencias": dest["assistencias"],
        "chutes_a_gol": dest["chutes"],
        "chances_criadas": dest["chances_criadas"],
        "faltas": dest["faltas"],
        "score": round(dest["score"], 1),
        # round(x, 1) arredonda para 1 casa decimal: 15.5 em vez de 15.499...
    })
```

13. Função main() — Ordenando e salvando os CSVs

```
# Ordena artilharia do maior para o menor número de gols
art = sorted(artilharia.items(), key=lambda x: -x[1]["gols"])
# sorted() retorna nova lista ordenada (não altera o original)
# .items() converte dicionário em lista de (chave, valor)
# key=lambda x: -x[1]["gols"] → ordena pelo número de gols (negativo = decrescente)
# Salva o CSV da artilharia
salvar_csv(
    os.path.join(pasta, "fifa_artilharia.csv"),
    ["pos", "jogador", "time", "gols"], # colunas
    [
        {"pos": i, "jogador": n, "time": v["time"], "gols": v["gols"]}
        for i, (n, v) in enumerate(art[:TOP_N], 1)
        # enumerate(art[:TOP_N], 1) → (1, item1), (2, item2)...
        # art[:TOP_N] → só os primeiros TOP_N itens
    ],
)

# O mesmo padrão se repete para faltas, chutes e destaque
flt = sorted(faltas_rank.items(), key=lambda x: -x[1]["faltas"])
cht = sorted(chutes_rank.items(), key=lambda x: -x[1]["chutes"])
# destaque_jogos já é uma lista, não precisa ordenar para salvar
```

Este script usa os endpoints `/topscorers` e `/topcards` da FIFA — dados oficiais e agregados. É mais simples que o anterior porque faz apenas 2 requisições no total (uma para cada endpoint).

14. Configuração e pesos da pontuação

```
# Pesos definidos como constantes no topo – fácil de ajustar
PESO_GOL = 4
PESO_ASSISTENCIA = 2
PESO_CHUTE_NO_ALVO = 1
PESO_CARTAO_AMARELO = -1
PESO_CARTAO_VERMELHO = -3
PESO_FALTA_COMETIDA = -0.5

# Usar constantes nomeadas é melhor do que escrever "4" direto no código
# porque fica claro o que significa cada número
```

15. Função: buscar() — Requisição genérica

Esta função centraliza o código de requisição. Em vez de repetir o mesmo bloco de código para `/topscorers` e `/topcards`, chamamos `buscar()` duas vezes com endpoints diferentes.

```
def buscar(endpoint, params=None):
    url = f"{BASE}/{endpoint}"
    # f-string monta a URL: "https://api.fifa.com/api/v3/topscorers"
    # Parâmetros padrão que vão em toda requisição
    params_base = {
        "idCompetition": ID_COMPETICAO, # "17" = Copa do Mundo
        "idSeason": ID_TEMPORADA, # "285023" = 2026
        "language": "pt", # resposta em português
        "count": 200, # máximo de resultados
    }
    # Se passamos parâmetros extras, adicionamos ao dicionário base
    if params:
        params_base.update(params)
        # .update() adiciona/sobrescreve chaves no dicionário
    resp = SESSAO.get(url, params=params_base, timeout=30)
    resp.raise_for_status()
    # raise_for_status() lança exceção se status != 200
    # (ex: 404 = não encontrado, 500 = erro no servidor)
    return resp.json().get("Results", [])
    # Retorna só a lista de resultados (o que nos interessa)
```

16. Função: calcular_pontuacao()

Separarmos o cálculo da pontuação em uma função própria tem duas vantagens: o código principal fica mais limpo, e podemos testar a fórmula isoladamente.

```
def calcular_pontuacao(gols, assist, alvo, amarelo, vermelho, faltas):
    return round(
        gols * PESO_GOL + # ex: 4 gols × 4 = 16 pontos
        assist * PESO_ASSISTENCIA + # ex: 2 assist × 2 = 4 pontos
        alvo * PESO_CHUTE_NO_ALVO + # ex: 9 chutes × 1 = 9 pontos
        amarelo * PESO_CARTAO_AMARELO + # ex: 0 amarelos × -1 = 0
        vermelho * PESO_CARTAO_VERMELHO + # ex: 0 vermelhos × -3 = 0
        faltas * PESO_FALTA_COMETIDA, # ex: 0 faltas × -0.5 = 0
        1, # arredonda para 1 casa decimal
    )
# Mbappé: 4×4 + 2×2 + 9×1 = 16+4+9 = 29.0 pontos
```

17. main() — Cruzando /topscorers com /topcards

Este é o trecho mais importante do script: cruzamos os dados de dois endpoints usando o ID do jogador como chave comum. A técnica é similar a um JOIN em SQL.

```
# 1. Busca os dois endpoints
scorers = buscar("topscorers") # 200 jogadores com stats ofensivas
cartoes = buscar("topcards") # 157 jogadores com cartões e faltas

# 2. Cria dicionário indexado pelo IdPlayer para busca rápida
# Em vez de percorrer a lista inteira para achar cada jogador (lento),
# acessamos em tempo constante: cartoes_por_id["229397"]
cartoes_por_id = {c["IdPlayer"]: c for c in cartoes}
# dict comprehension: { chave: valor for item in lista }

# 3. Para cada jogador dos topscorers, busca seus cartões
jogadores = []
for p in scorers:
    pid = p["IdPlayer"] # ex: "229397" (ID do Messi)
    # .get(pid, {}) → busca pelo ID; se não tiver cartão, retorna {}
    c = cartoes_por_id.get(pid, {})
    # Extrai cada campo com "or 0" para garantir número mesmo se vier None
    gols = p.get("Goals") or 0
    assist = p.get("Assists") or 0
    alvo = p.get("AttemptsOnTarget") or 0
    amarelo = c.get("YellowCards") or 0 # c pode ser {} se não tiver
    vermelho = c.get("RedCards") or 0
    faltas = c.get("FoulsCommitted") or 0
    pontuacao = calcular_pontuacao(gols, assist, alvo, amarelo, vermelho, faltas)
    jogadores.append({
        "jogador": texto(p.get("PlayerName")),
        "pais": p.get("IdCountry", ""),
        "gols": gols,
        "assistencias": assist,
        "participacoes_em_gol": gols + assist, # Goal Contributions
        "pontuacao": pontuacao,
        # ... outros campos
    })
```

18. main() — Ordenando por múltiplos critérios

```

# Ordena por 3 critérios em ordem de prioridade:
# 1º → maior Pontuação (negativo = decrescente)
# 2º → maior Participações em Gol (deempate)
# 3º → maior número de Gols (deempate final)
jogadores.sort(key=lambda x: (
    -x["pontuacao"],
    -x["participacoes_em_gol"],
    -x["gols"],
))
# sort() ordena no lugar (altera a lista original)
# A chave é uma tupla: Python compara elemento por elemento
# Adiciona o número de posição no ranking
for i, j in enumerate(jogadores, 1):
    j["pos"] = i # i começa em 1 (segundo argumento do enumerate)
# Salva os TOP_N primeiros no CSV
campos = ["pos", "jogador", "pais", "gols", "assistencias",
          "participacoes_em_gol", "chutes", "chutes_no_alvo",
          "cartoes_amarelos", "cartoes_vermelhos",
          "faltas_cometidas", "faltas_sofridas", "pontuacao"]
with open(saida, "w", newline="", encoding="utf-8-sig") as f:
    w = csv.DictWriter(f, fieldnames=campos)
    w.writeheader()
    w.writerows(jogadores[:TOP_N])
# jogadores[:TOP_N] → fatia da lista: só os primeiros TOP_N itens

```

19. Comparação entre os dois scripts

Característica	fifa_estatisticas_jogos.py	fifa_ranking_oficial.py
Endpoint usado	/timelines	/topscorers + /topcards
Nº de requisições	1 por jogo (64 no total)	Apenas 2
Tempo de execução	~20 segundos	~2 segundos
Dados por jogador	Eventos do jogo (bruto)	Estatísticas agregadas (oficial)
Destaque por jogo	Sim (score ponderado)	Não
Dados são oficiais?	Parcialmente (nossa fórmula)	Sim (dados diretos da FIFA)
Arquivo de entrada	fifa_jogos_completo.json	Nenhum (API direta)
Arquivo de saída	4 CSVs	1 CSV

20. Conceitos-chave para estudar mais

Conceito	Onde aparece	Para aprender mais
List comprehension	[j for j in lista if condição]	Python básico — listas
defaultdict	acumuladores de stats	Python collections
Regex (re.match)	extrair nome do jogador	Expressões regulares
f-strings	montar URLs e mensagens	Python strings
with open()	ler e salvar arquivos	Python I/O
.get(chave, padrão)	acessar JSON sem erro	Python dicionários
sorted() / .sort()	ordenar rankings	Python ordenação
enumerate()	loop com índice	Python built-ins
lambda	funções anônimas em sort/max	Python funções
try/except	tratamento de erros da API	Python exceções